Connections to Linear Algebra

From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

Linear Algebra in the Story

A neural network is, at its core, a sequence of matrix operations. Every forward pass, every parameter update, every attention computation is linear algebra. The table below maps standard course topics to their roles in modern AI.

Linear algebra topic	**Role in neural networks**
Matrix–vector multiplication	One layer of a feedforward network
Matrix–matrix multiplication	Batch forward pass; QKV projections
Dot product	Similarity score in attention
Row space, column space	What a layer can and cannot represent
Linear transformation	Each network layer as a map $\mathbb{R}^n \to \mathbb{R}^m$
Change of basis	Embedding and projection matrices
Transpose	$QK^\top$ in the attention formula
Softmax (row normalization)	Converting scores to probability weights
Weighted sum of vectors	Output of attention: AV
Rank and dimensionality	Bottleneck layers, low-rank approximations
Eigenvalues/eigenvectors	Optimization geometry, PCA for embeddings

Vectors and Matrices as the Computational Medium

Every object in a neural network is a vector or a matrix. Inputs are vectors. Parameters are matrices. Activations are vectors. The entire forward pass is a composition of matrix multiplications and elementwise nonlinearities.

For a single fully connected layer with weight matrix $W \in \mathbb{R}^{m\times n}$ and bias $\mathbf{b} \in \mathbb{R}^m$, the layer computes:

$$\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$$

where $\mathbf{x} \in \mathbb{R}^n$ is the input vector and $\mathbf{h} \in \mathbb{R}^m$ is the output vector. The weight matrix W is a linear transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$; the bias shifts the origin; the activation g introduces nonlinearity.

A deep network is a composition of such maps:

$$\mathbf{x} \xrightarrow{W^{(1)},\mathbf{b}^{(1)}} \mathbf{h}^{(1)} \xrightarrow{W^{(2)},\mathbf{b}^{(2)}} \mathbf{h}^{(2)} \xrightarrow{\cdots} \hat{\mathbf{y}}$$

Each arrow is a linear transformation followed by a nonlinearity. The composition is not linear — but each individual step is.

Key result. A neural network layer is a linear transformation followed by an elementwise nonlinearity. Without the nonlinearity, any composition of layers collapses to a single matrix multiplication — the network would have no more expressive power than a one-layer model. The nonlinearity is what makes depth useful.

The Embedding Matrix: Lookup as Linear Algebra

A vocabulary of V words is represented by an embedding matrix $E \in \mathbb{R}^{V \times d}$. Row i of E is the embedding vector for word i.

Looking up word i is equivalent to multiplying E by the one-hot vector $\mathbf{e}_i \in \mathbb{R}^V$ (all zeros except a 1 in position i):

$$E^\top \mathbf{e}_i = \text{row } i \text{ of } E$$

This is a change-of-basis operation: the one-hot representation uses the standard basis of $\mathbb{R}^V$; the embedding maps each basis vector to a learned vector in $\mathbb{R}^d$. The embedding matrix is the change-of-basis matrix from vocabulary space to semantic space.

Going deeper: geometry of embedding space. Because embeddings are learned to minimize prediction loss, words with similar contexts end up with similar embedding vectors. This means the geometry of $\mathbb{R}^d$ encodes semantic relationships. The famous result $\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}$ reflects the fact that the "royalty" direction and the "gender" direction are approximately orthogonal subspaces of the embedding space. Linear algebra students who have seen subspaces and projections are equipped to understand this structure precisely.

Attention as Linear Algebra

The attention mechanism is the heart of the transformer, and it is built entirely from matrix operations. Here we unpack each step.

The QKV Projections

Given a sequence of T tokens, each represented as a row vector in $\mathbb{R}^d$, we stack them into a matrix $X \in \mathbb{R}^{T \times d}$.

Three learned projection matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ transform X into query, key, and value matrices:

$$Q = XW_Q \in \mathbb{R}^{T \times d_k} \qquad K = XW_K \in \mathbb{R}^{T \times d_k} \qquad V = XW_V \in \mathbb{R}^{T \times d_k}$$

Row t of Q is the query vector for token t: what token t is looking for. Row t of K is the key vector for token t: what token t offers. Row t of V is the value vector for token t: what token t contributes to others.

The Similarity Matrix

The dot product $\mathbf{q}_t \cdot \mathbf{k}_s$ measures how relevant token s is to token t. Computing all T^2 pairwise dot products at once is a single matrix multiplication:

$$QK^\top \in \mathbb{R}^{T \times T}$$

Entry (t, s) of $QK^\top$ is $\mathbf{q}_t \cdot \mathbf{k}_s$ — the similarity score between token t's query and token s's key. The transpose is what aligns the dimensions for this multiplication.

Softmax Normalization

Dividing by $\sqrt{d_k}$ stabilizes the magnitudes, then softmax is applied row-wise to convert scores to weights that sum to 1:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T \times T}$$

Row t of A is a probability distribution over all tokens — the attention weights that token t places on every other token.

The Output

The attention output is a weighted combination of value vectors:

$$\text{Attention}(Q, K, V) = AV \in \mathbb{R}^{T \times d_k}$$

Row t of the output is $\sum_s A_{ts}\, \mathbf{v}_s$ — a weighted average of all value vectors, weighted by how much attention token t pays to each token s. This is a linear combination of rows of V, with weights given by row t of A.

Key result. The entire attention mechanism is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{XW_Q(XW_K)^\top}{\sqrt{d_k}}\right) XW_V$$

Every step is a matrix operation your linear algebra students already know: multiplication, transpose, row normalization, weighted sum. The only non-standard element is softmax, which is just a differentiable version of row normalization.

Historical note. The query-key-value framework was borrowed from information retrieval, where databases are searched by matching a query against a set of keys to retrieve associated values. In the original transformer (Vaswani et al., 2017), the Q/K/V abstraction made attention interpretable: each token asks a question (query), other tokens advertise their content (key), and contribute information if selected (value). The mathematical structure — dot-product similarity followed by softmax weighting — had appeared in earlier "soft attention" mechanisms (Bahdanau et al., 2015) for sequence translation. The 2017 paper showed that attention alone, without recurrence or convolution, was sufficient for state-of-the-art performance.

Student Activity: Attention by Hand

Background

The attention mechanism allows each token in a sequence to gather information from other tokens, weighted by relevance. Relevance is measured by dot-product similarity between learned projections of the token representations. In this activity you will compute one full pass of single-head attention on a three-token sequence.

The sequence is three tokens: **["the", "cat", "sat"]**, already converted to embedding vectors (rows of X).

Setup

Token embedding matrix ($T = 3$ tokens, $d = 4$ dimensions):

$$X = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \quad \leftarrow \text{rows are: "the", "cat", "sat"}$$

Projection matrices ($d = 4$ input, $d_k = 2$ head dimension):

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad W_V = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \\ 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

Part 1: Compute Q, K, V

Multiply X by each projection matrix. Each result is a 3×2 matrix.

Query matrix $Q = XW_Q$:

$$Q = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} __ & __ \\ __ & __ \\ __ & __ \end{pmatrix}$$

Key matrix $K = XW_K$:

$$K = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} __ & __ \\ __ & __ \\ __ & __ \end{pmatrix}$$

Value matrix $V = XW_V$:

$$V = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \\ 0.5 & 0 \\ 0 & 0.5 \end{pmatrix} = \begin{pmatrix} __ & __ \\ __ & __ \\ __ & __ \end{pmatrix}$$

Part 2: Compute the Similarity Matrix $QK^\top$

Multiply Q by $K^\top$. The result is a 3×3 matrix of raw similarity scores. Entry (i, j) is the dot product of row i of Q with row j of K.

$$S = QK^\top = \begin{pmatrix} __ & __ & __ \\ __ & __ & __ \\ __ & __ & __ \end{pmatrix}$$

Part 3: Scale and Apply Softmax

Scale each entry by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.707$:

$$\tilde{S} = \frac{S}{\sqrt{2}} = \begin{pmatrix} __ & __ & __ \\ __ & __ & __ \\ __ & __ & __ \end{pmatrix}$$

Softmax is applied row by row. For row i with entries $\tilde{s}_{i1}, \tilde{s}_{i2}, \tilde{s}_{i3}$:

$$A_{ij} = \frac{e^{\tilde{s}_{ij}}}{e^{\tilde{s}_{i1}} + e^{\tilde{s}_{i2}} + e^{\tilde{s}_{i3}}}$$

The pre-computed exponentials you will need (to 3 decimal places): $e^0 = 1.000$, $e^{0.707} \approx 2.028$, $e^{1.414} \approx 4.113$.

Apply softmax to each row of $\tilde{S}$ to get the attention weight matrix A:

$$A = \begin{pmatrix} ____ & ____ & ____ \\ ____ & ____ & ____ \\ ____ & ____ & ____ \end{pmatrix}$$

Check: each row of A should sum to 1.

Part 4: Compute the Output AV

Multiply A by V. The result is a 3×2 matrix. Row t of the output is the attention-weighted combination of all value vectors, as seen from token t.

$$\text{Output} = AV = \begin{pmatrix} ____ & ____ \\ ____ & ____ \\ ____ & ____ \end{pmatrix}$$

Part 5: Reflection

1. Look at row 1 of A (attention weights for `the''). Which token does` the'' attend to most strongly? Does this make linguistic sense?

2. The diagonal entries A_{11}, A_{22}, A_{33} represent each token's attention to itself. Are they the largest entries in their rows? What would it mean if a token attended primarily to itself?

3. The value matrix V is a scaled-down version of the input X (every entry is halved). What effect does this have on the output? In a real transformer, why might W_V be learned rather than fixed?

What you just computed. You performed one full pass of single-head scaled dot-product attention: three matrix multiplications, one transpose, one scaling, one row-wise softmax, and one final matrix multiplication. In GPT-3, this computation runs across 96 layers, each with 96 attention heads, on sequences of up to 2048 tokens, with $d_k = 128$. The matrices are larger; the operations are identical.

Answer Key

Part 1: QKV matrices

$$Q = XW_Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix} \qquad K = XW_K = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix} \qquad V = XW_V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}$$

Verification for Q, row 1: $[1, 0, 1, 0] \cdot W_Q$: column 1: $1(1) + 0(0) + 1(1) + 0(0) = 2$; column 2: $1(0) + 0(1) + 1(0) + 0(1) = 0$. □

Part 2: Similarity matrix

$$S = QK^\top = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 4 & 2 \\ 4 & 0 & 2 \\ 2 & 2 & 2 \end{pmatrix}$$

Entry $(1, 2)$: $\mathbf{q}_1 \cdot \mathbf{k}_2 = [2, 0] \cdot [2, 0] = 4$. `the'' findscat`'' highly relevant.

Part 3: Scaled similarity and softmax

$$\tilde{S} = \frac{S}{\sqrt{2}} \approx \begin{pmatrix} 0 & 2.828 & 1.414 \\ 2.828 & 0 & 1.414 \\ 1.414 & 1.414 & 1.414 \end{pmatrix}$$

Applying softmax row by row (using $e^0 = 1$, $e^{1.414} \approx 4.113$, $e^{2.828} \approx 16.92$):

Row 1: denominator $= 1 + 16.92 + 4.113 = 22.033$

$$A_{1\cdot} \approx (0.045,\ 0.768,\ 0.187)$$

Row 2: denominator $= 16.92 + 1 + 4.113 = 22.033$

$$A_{2\cdot} \approx (0.768,\ 0.045,\ 0.187)$$

Row 3: denominator $= 4.113 + 4.113 + 4.113 = 12.339$

$$A_{3\cdot} \approx (0.333,\ 0.333,\ 0.333)$$

$$A \approx \begin{pmatrix} 0.045 & 0.768 & 0.187 \\ 0.768 & 0.045 & 0.187 \\ 0.333 & 0.333 & 0.333 \end{pmatrix}$$

Each row sums to 1. □

Part 4: Output AV

$$AV \approx \begin{pmatrix} 0.045 & 0.768 & 0.187 \\ 0.768 & 0.045 & 0.187 \\ 0.333 & 0.333 & 0.333 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}$$

Row 1: $[0.045(1) + 0.768(0) + 0.187(0.5),\ \ 0.045(0) + 0.768(1) + 0.187(0.5)] \approx [0.139,\ 0.862]$

Row 2: $[0.768(1) + 0.045(0) + 0.187(0.5),\ \ 0.768(0) + 0.045(1) + 0.187(0.5)] \approx [0.862,\ 0.139]$

Row 3: $[0.333(1) + 0.333(0) + 0.333(0.5),\ \ 0.333(0) + 0.333(1) + 0.333(0.5)] \approx [0.500,\ 0.500]$

$$\text{Output} \approx \begin{pmatrix} 0.139 & 0.862 \\ 0.862 & 0.139 \\ 0.500 & 0.500 \end{pmatrix}$$

Part 5 discussion notes:

Row 1 of A: `the'' attends most strongly to`cat'' (weight 0.768). Linguistically, determiners attach to the nouns they modify, so this is a meaningful result — the model has learned, through these particular weight matrices, to route `the'' to-ward`cat''.

The diagonal entries ($A_{11} = 0.045$, $A_{22} = 0.045$, $A_{33} = 0.333$) are not the largest entries for rows 1 and 2. `the'' and`cat'' attend more strongly to each other than to themselves. Token 3 ("sat'') attends equally to all three tokens because its query and key structure produces identical scores — a uniform distribution is the result.

The scaled W_V halves all values in V, which halves all entries in the output. In a real transformer, W_V is learned so the model can control what information each token contributes, independently of what it seeks (query) or advertises (key).

Classroom Connections: Linear Algebra Topic Map

Matrix Multiplication

Every layer of a neural network is a matrix multiplication. The standard course treatment — row times column, dimensions must match, $(AB)_{ij} = \sum_k A_{ik}B_{kj}$ — is exactly the operation performed billions of times per second during training.

The shape constraint ($m \times n$ times $n \times p$ gives $m \times p$) has a direct interpretation: the inner dimension n is the number of features flowing between layers. When you teach students to check conformability, you are teaching them to reason about information flow in neural networks.

Linear Transformations

Each network layer is a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ defined by $T(\mathbf{x}) = W\mathbf{x}$. The standard course results apply directly:

The **null space** of W is the set of inputs that produce zero output — directions in input space that the layer ignores entirely.

The **column space** of W is the set of possible outputs — the reachable representations in the next layer. A layer with $\text{rank}(W) < m$ is a bottleneck: it compresses information.

Rank and expressiveness are directly related. A weight matrix of rank r can represent at most r linearly independent directions in output space. This is why neural networks need many neurons per layer to be expressive.

Dot Products and Geometry

The attention score $\mathbf{q}_t \cdot \mathbf{k}_s$ is a dot product. Your students learn that the dot product measures the projection of one vector onto another: large when vectors point in similar directions, zero when orthogonal, negative when opposing.

In attention, this geometric intuition is directly meaningful: tokens with aligned query-key vectors attend to each other; tokens with orthogonal query-key vectors ignore each other. The geometry your students learn in the abstract is the mechanism by which language models understand context.

Change of Basis

The embedding matrix E maps one-hot vocabulary vectors to dense embedding vectors. This is a change of basis from the standard basis of $\mathbb{R}^V$ (one dimension per word) to a learned basis of $\mathbb{R}^d$ (directions in semantic space). The QKV projection matrices are further changes of basis within the embedding space, transforming token representations into query, key, and value spaces where different notions of similarity are operative.

Going deeper: low-rank structure. GPT-3 attention heads use $d_k = 128$ with $d = 12288$. The projection $W_Q \in \mathbb{R}^{12288 \times 128}$ has rank at most 128 — it projects the full embedding space onto a 128-dimensional subspace for the purpose of computing similarity. This low-rank structure is not a limitation; it is a design choice that forces each attention head to focus on a low-dimensional notion of relevance. Recent work (Hu et al., 2021 — LoRA) exploits low-rank structure to fine-tune large models efficiently by approximating weight updates as $\Delta W = AB$ where $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$ with $r \ll d$.

Eigenvalues and Optimization

Eigenvalues appear in neural network training through the Hessian matrix $H = \nabla^2 L$ of the loss function. The eigenvalues of H determine the curvature of the loss surface: large eigenvalues correspond to directions of steep curvature where gradient descent must take small steps; small eigenvalues correspond to flat directions where progress is slow.

The ratio of the largest to smallest eigenvalue of H — the *condition number* — determines how difficult the optimization is. A poorly conditioned loss surface is why neural network training requires careful learning rate choices and why techniques like batch normalization (which improves conditioning) matter.

For instructors. Linear algebra is not a supporting tool for neural networks — it *is* neural networks. Every forward pass is a sequence of matrix multiplications. Every attention computation is a structured sequence of matrix products and normalizations. Every weight update is a matrix update. Students who understand linear transformations, dot products, matrix multiplication, and rank have the complete mathematical vocabulary for understanding how these systems work. The gap between a linear algebra course and a working understanding of transformer architecture is smaller than most instructors realize.